

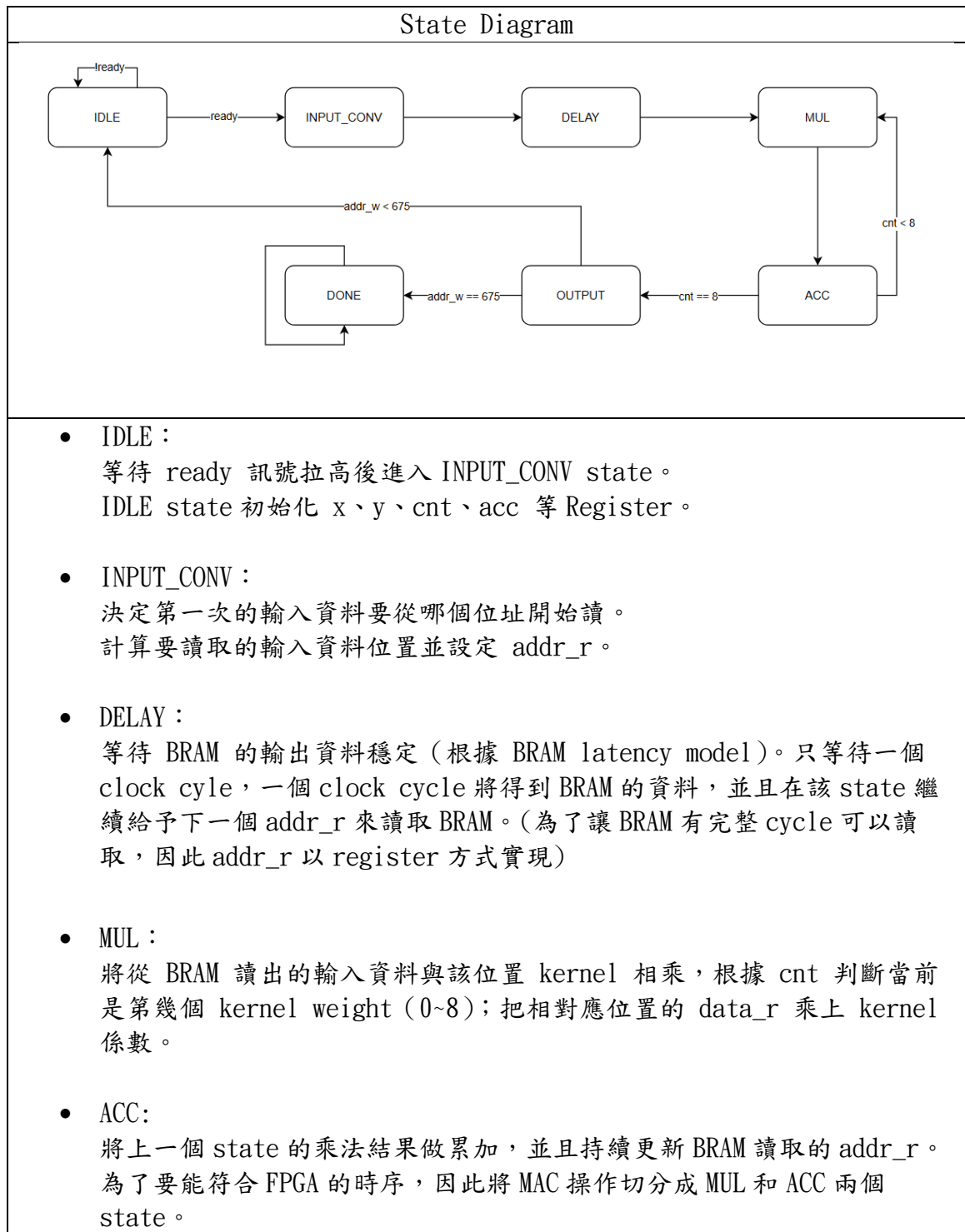
DLIC HW10

系級:資訊甲 114

學號:E34106010

姓名:黃偉峰

1. CONV 電路設計



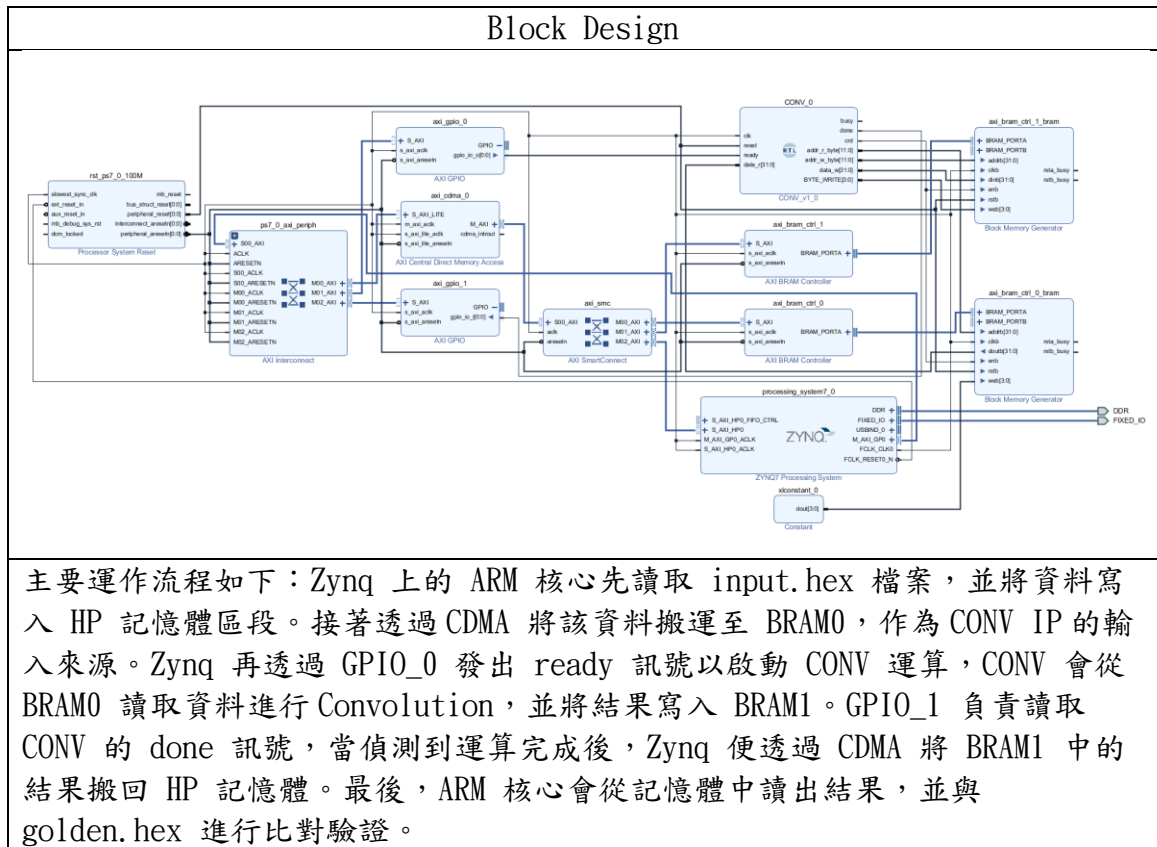
- OUTPUT :

將累加後的卷積結果寫入 data_w，並更新 addr_w。將 acc 值寫入輸出的記憶體位置。若 addr_w == 675 進入 DONE；否則回到 INPUT_CONV 處理下一個 CONV 操作。

- DONE:

表示整張圖的卷積處理結束。拉高 done 訊號，通知處理完成。

2. CONV 系統設計



3. 軟體設計流程

```
# === Address Mapping ===
CDMA_BASE = 0x7E200000
DDR_SRC = 0x20000000
DDR_DST = 0x2000C000
BRAM0_ADDR = 0xC0000000
BRAM1_ADDR = 0xC2000000
GPIO0_ADDR = 0x41200000
GPIO1_ADDR = 0x41210000
```

```
in_bytes = 784 * 4
out_bytes = 676 * 4
```

```
# === MMIO ===
cdma = MMIO(CDMA_BASE, 0x1000)
ddr_src = MMIO(DDR_SRC, in_bytes)
ddr_dst = MMIO(DDR_DST, in_bytes)
bram0 = MMIO(BRAM0_ADDR, 8000)
bram1 = MMIO(BRAM1_ADDR, 8000)
GPIO0 = MMIO(GPIO0_ADDR, 64)
GPIO1 = MMIO(GPIO1_ADDR, 64)
```

這段程式定義了 CDMA、DDR、BRAM 及 GPIO 的 MMIO，並利用 MMIO 進行對應模組的記憶體存取初始化，為後續資料搬運與控制操作作好準備。

```
# === Step 1: Load HEX file to DDR memory ===
with open("input.hex", "r") as f:
    lines = f.readlines()

data = [int(line.strip(), 16) for line in lines]
byte_len = len(data) * 4

for i, val in enumerate(data):
    ddr_src.write(i * 4, val)
```

透過 Zynq 上的 ARM CPU 將 input.hex 的資料讀取到 HP 的記憶體區段。

```
# === Step 2: DDR -> BRAM0 ===
cdma.write(0x00, 0x04)
cdma.write(0x18, DDR_SRC)
cdma.write(0x20, BRAM0_ADDR)
cdma.write(0x28, in_bytes)

# Wait for CDMA transfer to complete
while (cdma.read(0x04) & 0x2) == 0:
    pass
```

使用 CDMA 將 HP 記憶體區段的資料搬運到 BRAM0 內部。

```
# --- Step 3: Send ready=1 via GPIO and wait for done=1 ---

GPIO0.write(0x4, 0x0)
GPIO1.write(0x4, 0x1)
GPIO0.write(0x0, 0x1)
while True:
    val = GPIO1.read(0x0)
    if val & 0x1: # bit 1 = done
        break
```

透過 Zynq CPU 寫入 GPIO_0 訊號以供 CONV IP 當作 ready 訊號來做開始計算的依據，並且使用 GPIO_1 持續讀取 CONV IP 的 done 訊號，若為 1 CONV 運算完成。

```
# --- Step 4: BRAM1 -> DDR ---

cdma.write(0x00, 0x04)
cdma.write(0x18, BRAM1_ADDR)          # source: result from conv
cdma.write(0x20, DDR_DST)              # destination: back to DDR
cdma.write(0x28, out_bytes)             # result size
while (cdma.read(0x04) & 0x2) != 0:
    pass
```

使用 CDMA 將 BRAM1 內部的資料搬運到 Zynq 的 HP 記憶體區段。

```
# --- Step 5: Compare with golden.hex ---

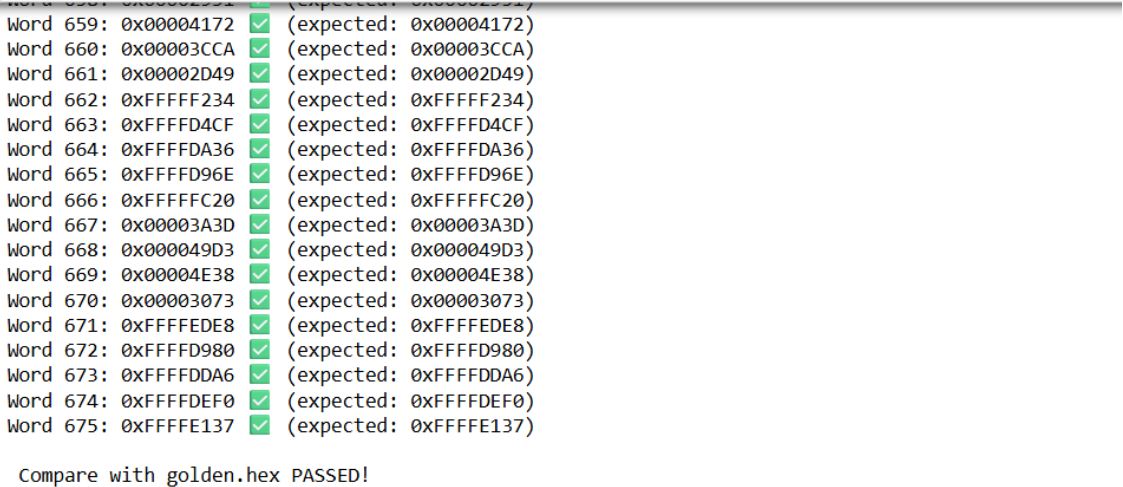
with open("golden.hex", "r") as f:
    golden = [int(line.strip(), 16) for line in f]

success = True
print(f"\n Verifying {len(golden)} words against golden.hex:")
for i, expected in enumerate(golden):
    val = ddr_dst.read(i * 4)
    ok = (val == expected)
    print(f"Word {i:3}: 0x{val:08X} {'✅' if ok else '❌'} (expected: 0x{expected:08X})")
    if not ok:
        success = False

if success:
    print("\n Compare with golden.hex PASSED!")
else:
    print("\n Compare FAILED. Mismatch found.")
```

最後將 golden.hex 的值去跟從 BRAM1 搬運回來的值做比較，確認 CONV 運算是否正常。

4. 比對結果

自行設計的軟體版本	
	
助教提供軟體版本	
	

5. 作業學習心得

這次作業我覺得具有一定難度，需要將過去幾次作業所學的內容整合起來完成整體設計。在整個設計過程中，我卡關最久的部分是 CONV IP 與 BRAM 之間的讀取與寫入操作。主要是因為 BRAM 採用 Byte Addressable 的設計，但我一開始沒有注意到這點，錯誤地以為是 Word Addressable，導致在設計 CONV 讀寫 BRAM 的邏輯時發生資料錯誤。後來在查閱文件與實際模擬波形後，才突然發現這個問題。

另一個遇到的問題是在 Timing 上。最初我設計的 CONV IP 是在單一 cycle 內完成一次 MAC 運算，但實際運作後發現部分結果正確、部分錯誤。這使

我意識到可能是 Timing 達不到所產生的問題，因為在 FPGA 上不像 ASIC 一樣 FPGA 是使用已有的硬體去做組合，所以在 timing 上通常會相較 ASIC 設計上更嚴格，因此我隨即將設計修改為一個 cycle 只執行乘法，下一個 cycle 再進行累加。更改後所有結果皆正確，也順利完成驗證。